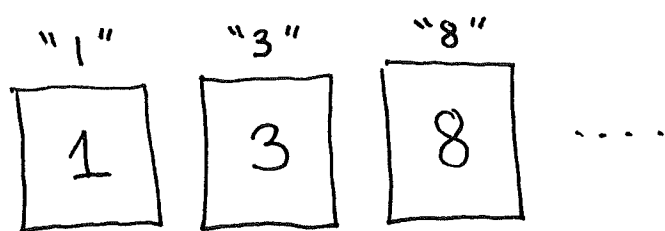


Neural networks in machine learning:

"Supervised" learning: Given training data, $\{x^u, y^u\}$ build a predictive model of y from x . $u=1, \dots, P$.

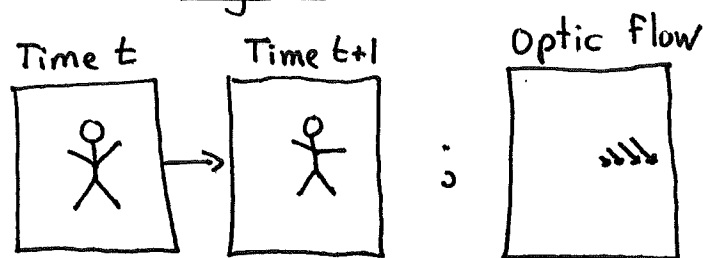
Classification:



X : Image of handwritten digit

y : Label indicating number (Discrete)

Regression:

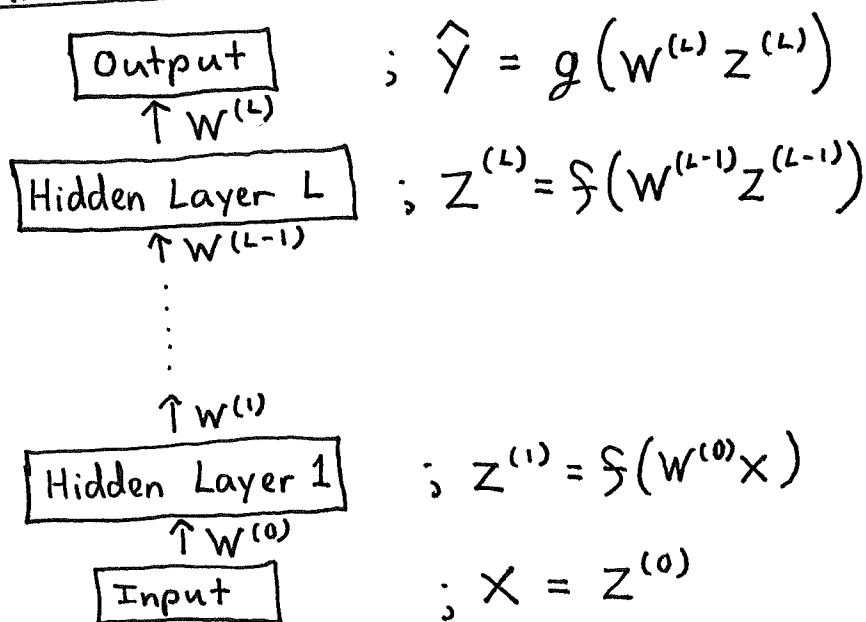


X : Image sequence

y : Image of optic flow (Continuous)

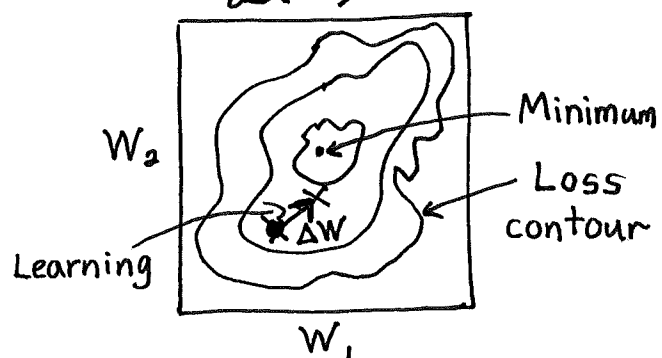
Three elements of algorithm design: architecture; loss function; learning rule

Architecture:



"Deep neural network"

Loss function:



C : "Cross entropy" ; R : "Squared error"

Learning rule:

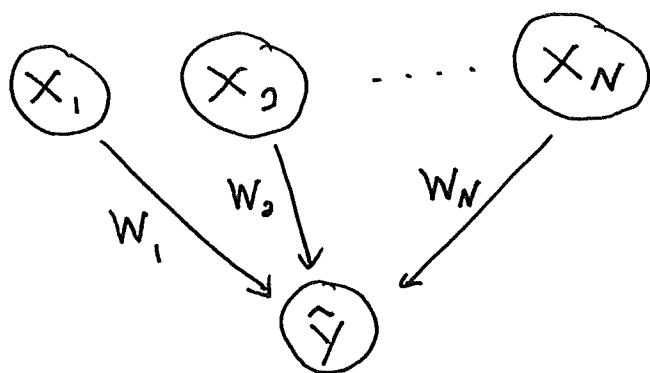
$$\Delta W_{ij}^{(l)} (\{x^u, y^u\}, w^{(0)}, w^{(1)}, \dots, w^{(l)})$$

Should be chosen to minimize loss.

"Gradient descent"

Linear regression:

Architecture:

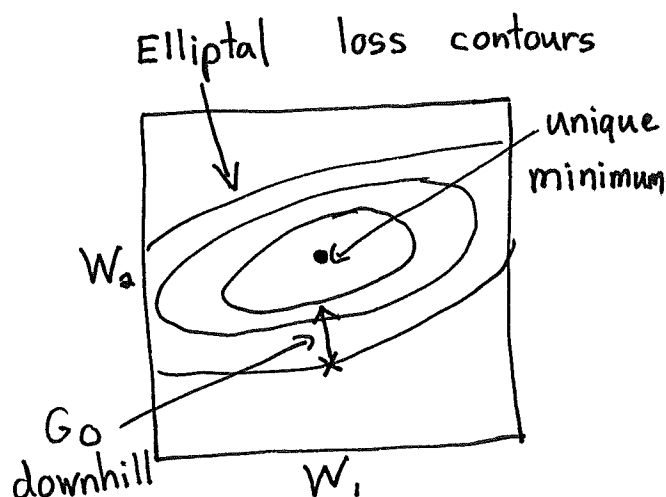


$$\hat{y} = Wx = \sum_{i=1}^N w_i x_i$$

Loss function:

$$\begin{aligned} \mathcal{L}(W) &= \frac{1}{P} \sum_{m=1}^P (y_m - \hat{y}_m)^2 \\ &= \frac{1}{P} \sum_{m=1}^P (y_m - \sum_{i=1}^N w_i x_{im})^2 \end{aligned}$$

Quadratic function of W



Gradient descent learning:

$$\mathcal{L}(W + \Delta W) = \underbrace{\mathcal{L}(W) + \sum_{i=1}^N \frac{\partial \mathcal{L}(W)}{\partial w_i} \Delta w_i}_{\text{Leading Taylor series expansion}} + \underbrace{O((\Delta W)^2)}_{\text{Small / Ignore}}$$

$$\Delta \mathcal{L} = \underbrace{\nabla_W \mathcal{L}}_{\text{Gradient}} \cdot \Delta W = \underbrace{\|\nabla_W \mathcal{L}\|}_{\text{Fixed}} \underbrace{\|\Delta W\| \cos \theta}_{\text{minimize}}$$

$$\Delta W = -\underbrace{\eta}_{\text{Learning rate}} \nabla_W \mathcal{L}$$

Learning rule:

$$\begin{aligned} \Delta w_i &= -\eta \frac{\partial \mathcal{L}}{\partial w_i} = -\eta \frac{1}{P} \sum_{m=1}^P 2(y_m - \hat{y}_m)(-x_{im}) \\ &= \frac{2\eta}{P} \sum_{m=1}^P e_m x_{im} \quad (\text{Three-factor form}) \end{aligned}$$

Equivalently:

$$\Delta W_i = \frac{2\eta}{p} \sum_{\mu=1}^p \left(\underbrace{y_{\mu} x_{i\mu}}_{\text{Hebbian}} - \underbrace{\sum_{j=1}^N W_j x_{j\mu} x_{i\mu}}_{\text{stabilization}} \right)$$

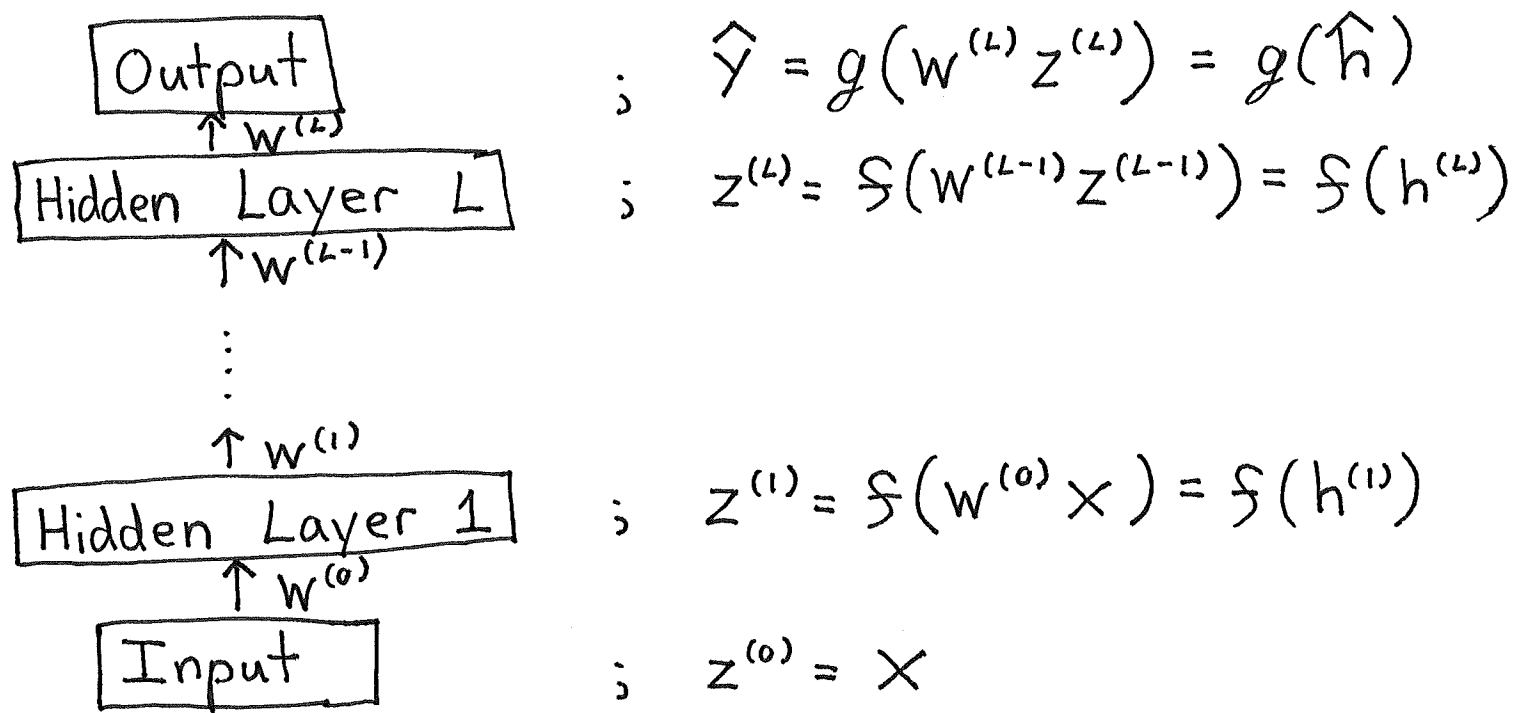
Classic linear regression formula

$$\Delta W = 0 \Rightarrow \frac{2\eta}{p} (X Y - X X^T W) = 0$$

$$\Rightarrow \underbrace{W}_{N \times 1} = \underbrace{(X X^T)^{-1}}_{N \times N} \underbrace{X Y}_{N \times 1}$$

Backpropagation algorithm:

We return to the deep network architecture.



Deep linear regression:

Suppose $g(x) = S(x) = x$. Then

$$\hat{y} = W^{(L)} z^{(L)} = W^{(L)} W^{(L-1)} z^{(L-1)} = W^{(L)} W^{(L-1)} \dots W^{(0)} x$$

Input-output mapping is linear

$$\hat{y} = M x; \quad M = \prod_{\ell=0}^L W^{(\ell)}$$

Learning rule and learning dynamics are complex.

For regression

$$\mathcal{L} = \frac{1}{P} \sum_{\mu=1}^P \mathcal{L}_{\mu}$$

$$\mathcal{L}_{\mu} = (y_{\mu} - \hat{y}_{\mu})^2 = e_{\mu}^2$$

As before, we consider gradient-descent learning

$$\Delta W_{ij}^{(\ell)} = -\eta \frac{\partial \mathcal{L}}{\partial W_{ij}^{(\ell)}}$$

We derive the derivative by repeatedly applying the chain rule from calculus.

$$\begin{aligned}
 \frac{\partial \mathcal{L}}{\partial W_{ij}^{(L)}} &= \frac{1}{P} \sum_{\mu} \frac{\partial d_{\mu}}{\partial W_{ij}^{(L)}} \\
 &= \frac{1}{P} \sum_{\mu} \frac{\partial d_{\mu}}{\partial \hat{y}_{\mu}} \frac{\partial \hat{y}_{\mu}}{\partial W_{ij}^{(L)}} \\
 &= \frac{1}{P} \sum_{\mu} \underbrace{\frac{\partial d_{\mu}}{\partial \hat{y}_{\mu}}}_{-2e_{\mu}} \sum_{k_L=1}^{N_L} \underbrace{\frac{\partial \hat{y}_{\mu}}{\partial z_{k_L \mu}^{(L)}}}_{\frac{1}{W_{k_L}^{(L)}}} \sum_{k_{L-1}=1}^{N_{L-1}} \underbrace{\frac{\partial z_{k_L \mu}^{(L)}}{\partial z_{k_{L-1} \mu}^{(L-1)}}}_{\frac{1}{W_{k_L, k_{L-1}}^{(L-1)}}} \\
 &\quad \dots \sum_{k_{L+1}=1}^{N_{L+1}} \underbrace{\frac{\partial z_{k_{L+2} \mu}^{(L+2)}}{\partial z_{k_{L+1} \mu}^{(L+1)}}}_{\frac{1}{W_{k_{L+2}, k_{L+1}}^{(L+1)}}} \underbrace{\frac{\partial z_{k_{L+1} \mu}^{(L+1)}}{\partial W_{ij}^{(L)}}}_{\delta_{i, k_{L+1}} Z_{j \mu}^{(L)}}
 \end{aligned}$$

The top equation is general and will be reused soon. The linearity of the network and regression loss lead to the lower expression and simple final result.

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(L)}} = \frac{-2}{P} \sum_{\mu} \left(W^{(L+1)T} W^{(L+2)T} \dots W^{(L)T} e_{\mu} \right)_i Z_{j \mu}^{(L)}$$

This is similar to linear regression, but the post-synaptic error becomes a "backpropagated" error.

Deep nonlinear regression:

Next suppose that $f(x) \neq x$, $g(x) = x$, $d_{\mu} = e_{\mu}^2$. Most of the above analysis still holds.

However, we apply the chain rule another time in each layer to account for the nonlinearity.

$$\frac{\partial z_{k_{n+1}\mu}^{(n+1)}}{\partial z_{k_n\mu}^{(n)}} = \frac{\partial z_{k_{n+1}\mu}^{(n+1)}}{\partial h_{k_{n+1}\mu}^{(n+1)}} \frac{\partial h_{k_{n+1}\mu}^{(n+1)}}{\partial z_{k_n\mu}^{(n)}} = \mathcal{F}'(h_{k_{n+1}\mu}^{(n+1)}) W_{k_{n+1}k_n}^{(n)}$$

The $\mathcal{F}'$ terms make the formulae ugly, but the basic concept is the same. An error signal is backpropagated with $W^{(L)T}$, multiplied with the derivative of the activation function at the L^{th} layer, backpropagated with $W^{(L-1)T}$, and so on.

(Lots of data) + (Lots of layers) = (Revolution in machine learning, circa 2012)

<u>Hinton, Krizersky</u>	<u>Sutskever</u>
Founded Gatsby Computational Neuroscience Unit	Invented ChatGPT a decade later